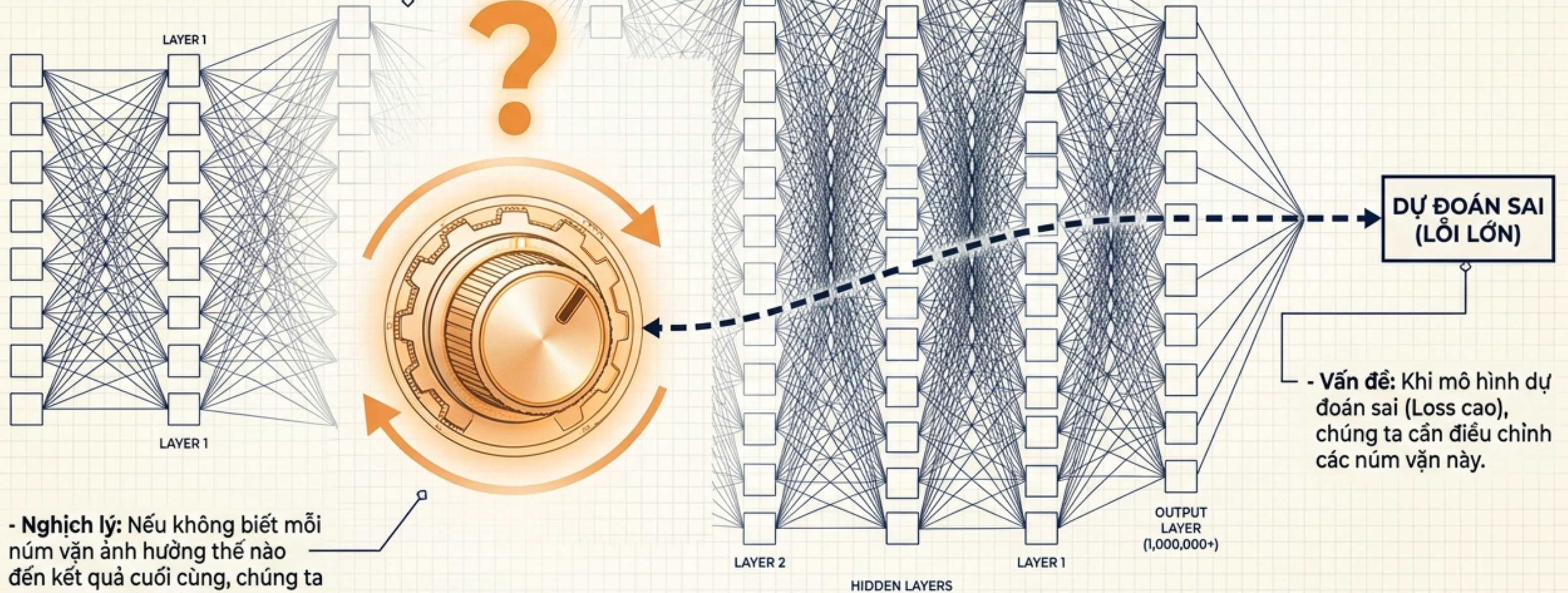


THE CRISIS: LÀM SAO ĐIỀU CHỈNH MỘT CỔ MÁY CÓ HÀNG TRIỆU NÚM VẶN?

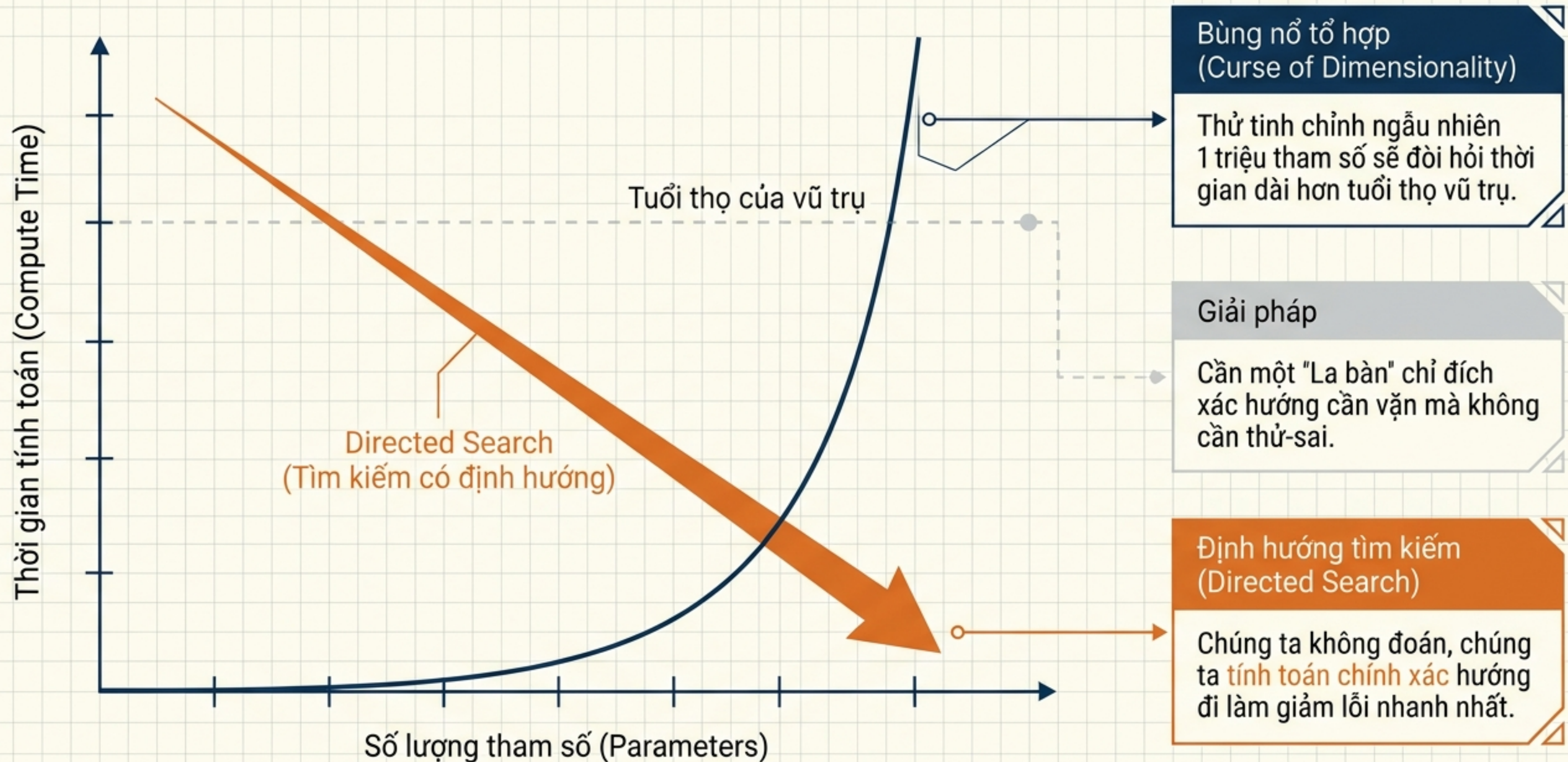
- **Khái niệm:** Mô hình học máy thực chất là một cỗ máy toán học với hàng triệu, thậm chí hàng tỉ tham số (Weights/Biases).



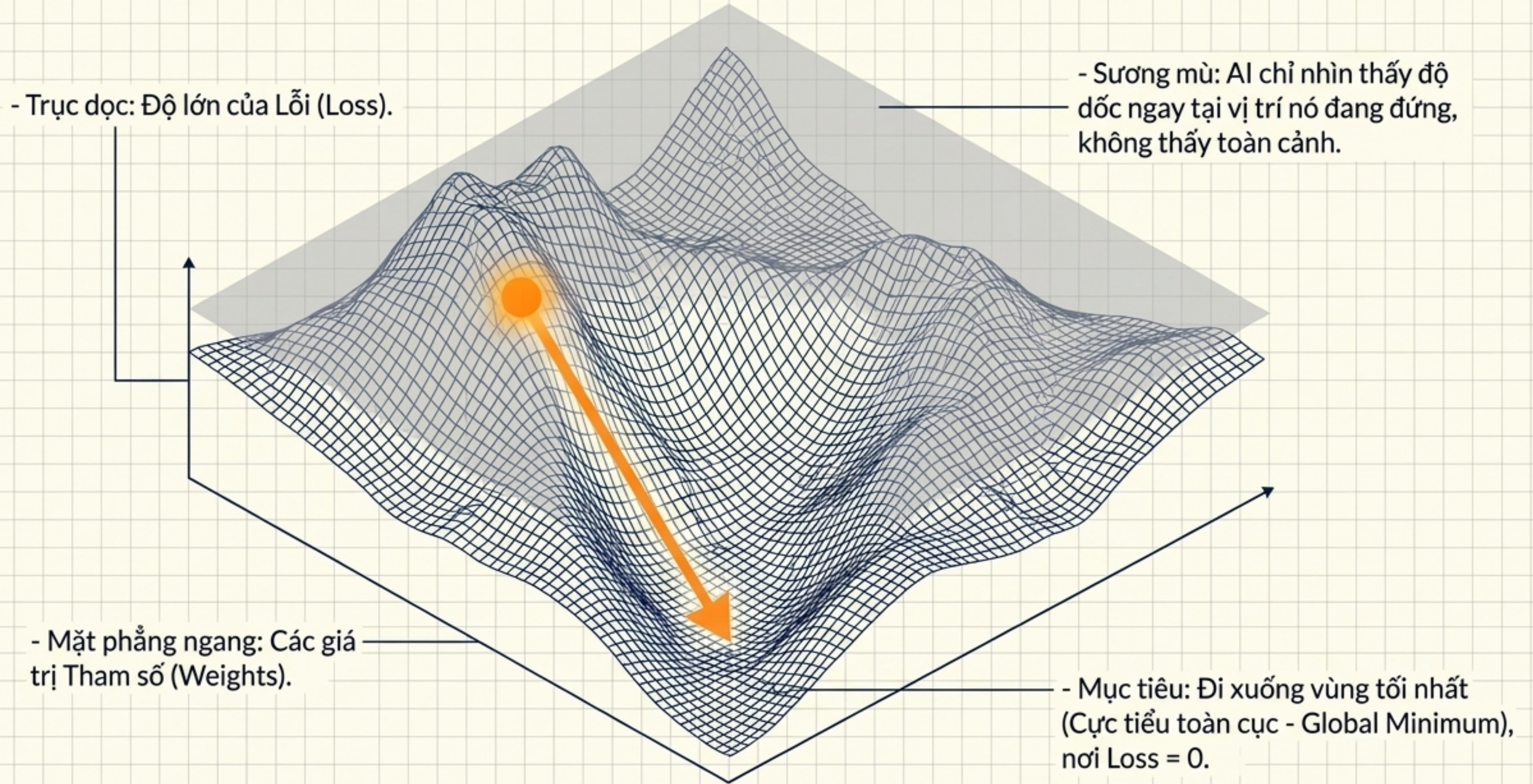
- **Nghịch lý:** Nếu không biết mỗi nút vặn ảnh hưởng thế nào đến kết quả cuối cùng, chúng ta hoàn toàn mù đường.

- **Vấn đề:** Khi mô hình dự đoán sai (Loss cao), chúng ta cần điều chỉnh các nút vặn này.

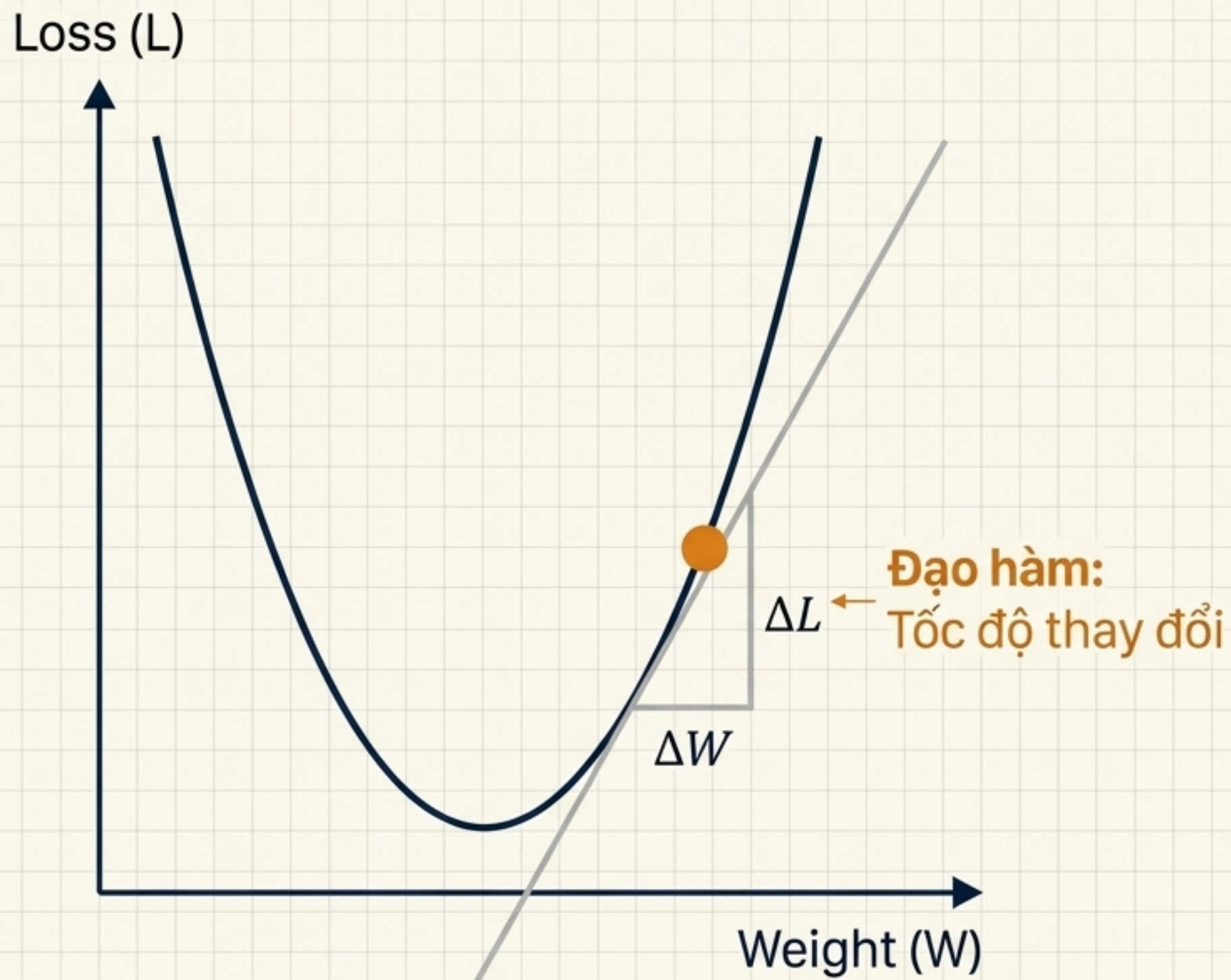
THE WHY: TẠI SAO PHƯƠNG PHÁP “THỬ & SAI” LẠI BẤT LỰC?



INTUITION: ĐỊA HÌNH HÀM MẤT MÁT (LOSS SURFACE) & SƯƠNG MÙ



THE FOUNDATION: ĐẠO HÀM (DERIVATIVE) – THƯỚC ĐO ĐỘ DỐC



1. Ý nghĩa hình học

Đạo hàm là độ dốc của đường tiếp tuyến tại một điểm.

2. Đo lường sự thay đổi

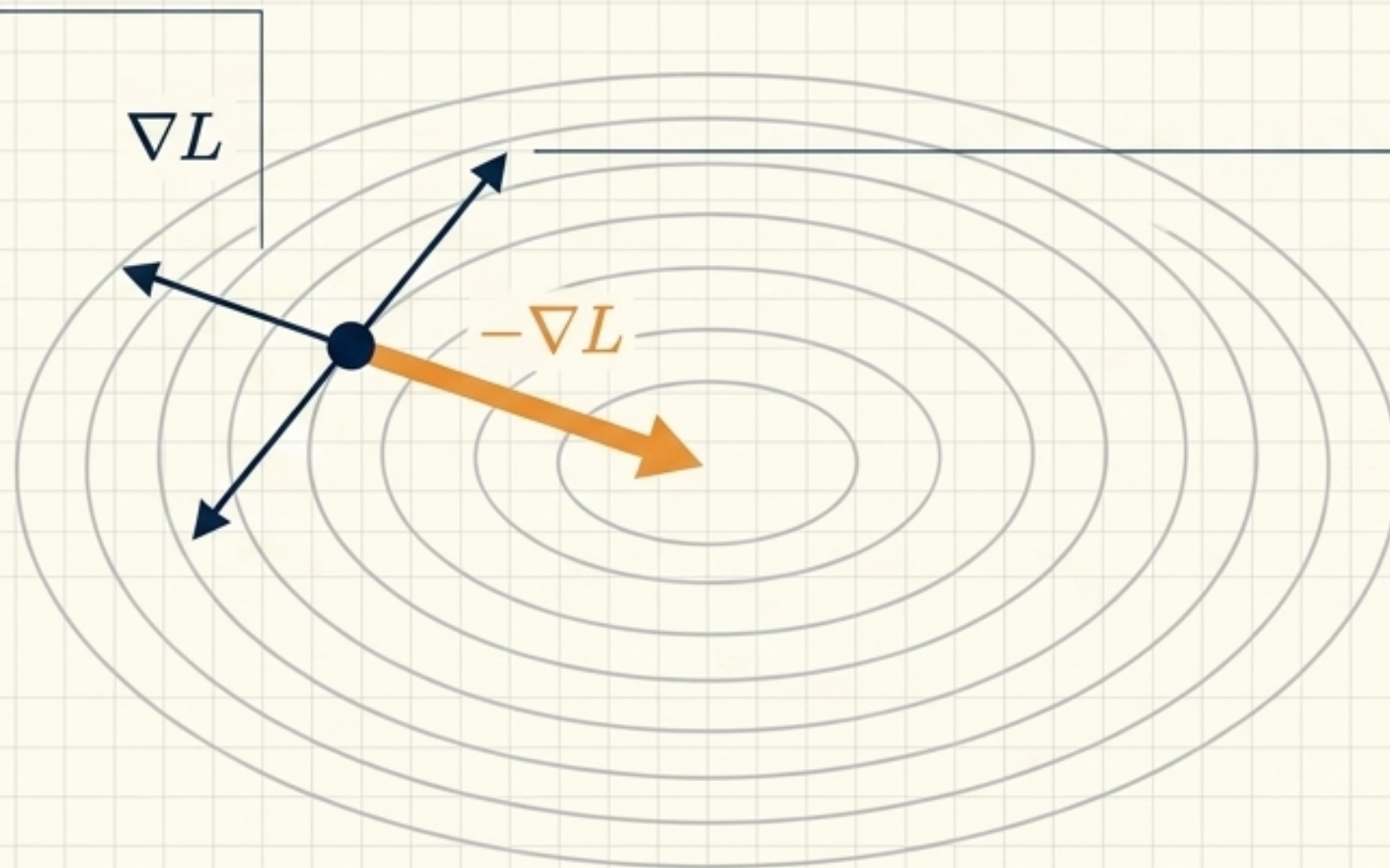
$\frac{dL}{dW} = -4$ nghĩa là: Nếu tăng W lên 1 đơn vị, Loss sẽ giảm 4 đơn vị.

3. Điểm hội tụ

Khi Đạo hàm = 0, đường tiếp tuyến nằm ngang $\rightarrow$ Đã chạm đáy thung lũng (Cực trị).

THE CONCEPT: GRADIENT – CHIẾC LA BÀN TRONG KHÔNG GIAN N-CHIỀU

- Gradient (∇L): Là một vector tập hợp mọi đạo hàm riêng của tất cả tham số.
- Đặc tính toán học: Gradient LUÔN chỉ hướng dốc LÊN nhanh nhất.



- Tại sao có dấu trừ?

Thuật toán tối ưu hóa yêu cầu AI phải đi NGƯỢC hướng Gradient ($-\nabla L$) để đi xuống thung lũng tìm điểm Loss thấp nhất.

THE ENGINE: THUẬT TOÁN GRADIENT DESCENT (HẠ GRADIENT)

$$W_{new} = W_{old} - \eta \cdot \nabla L$$

Vị trí hiện tại -> Vị trí bước tiếp theo.

La bàn (hướng dốc nhất).

Learning Rate (Độ dài của bước chân).

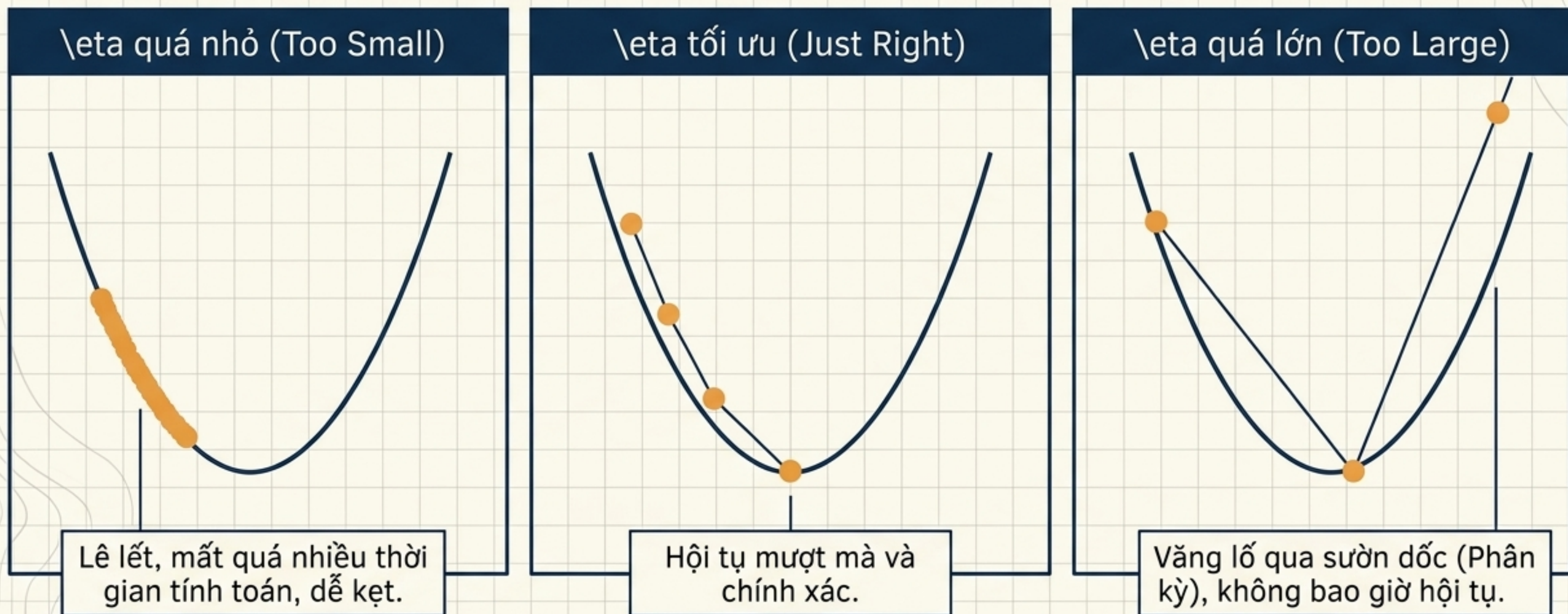
1. Đo độ dốc (∇L)

2. Nhân với η

3. Bước đi
(Cập nhật W)

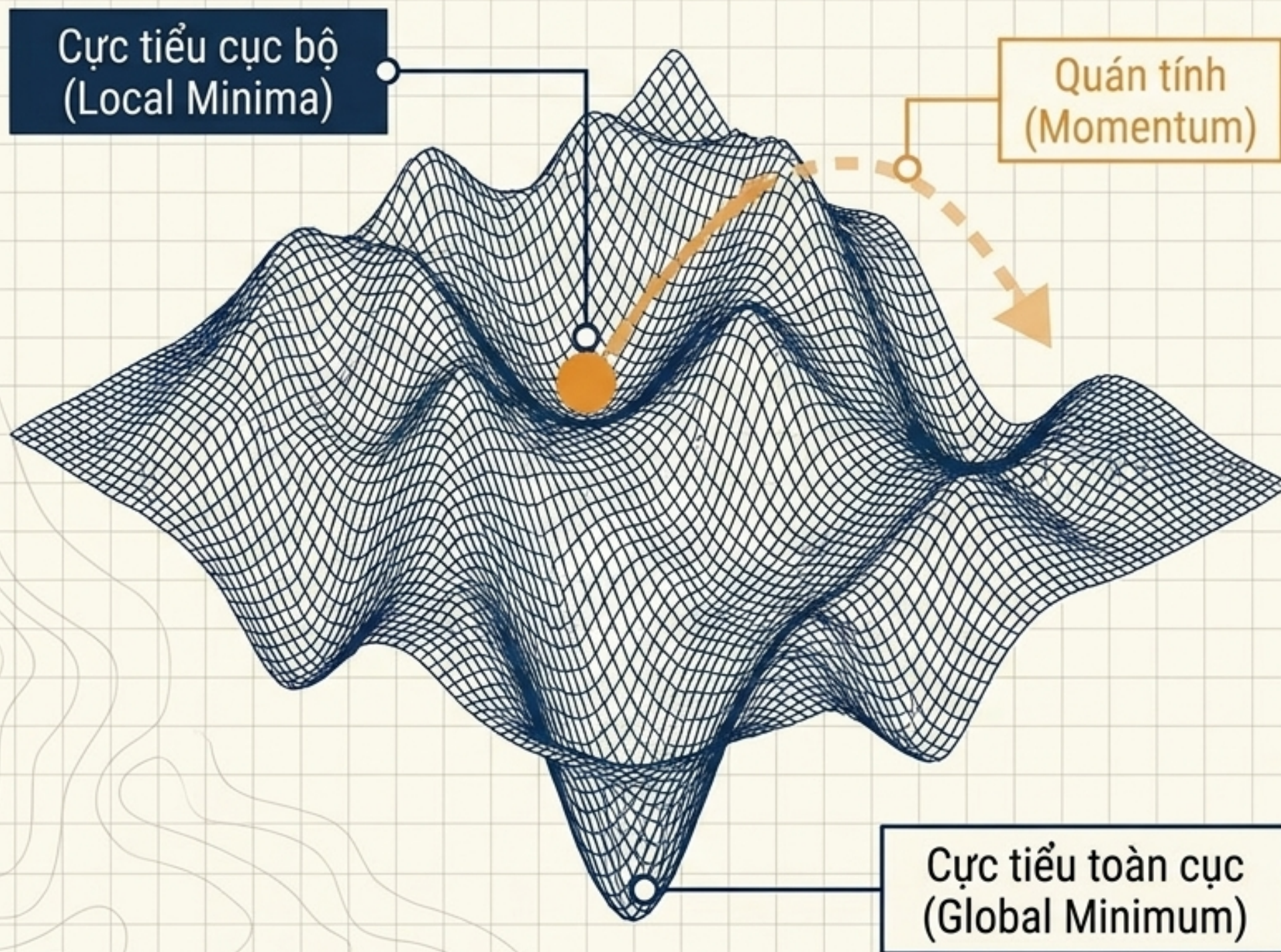
- **Bản chất:** Động cơ cốt lõi giúp hàng tỉ tham số tự động xoay đúng hướng.
- **Vòng lặp liên tục:** Quá trình (Đo -> Bước -> Cập nhật) được lặp lại hàng triệu lần.

TRADE-OFF 1: NGHỆ THUẬT CHỌN ĐỘ DÀI BƯỚC CHÂN (LEARNING RATE η)



- Thực tế: AI Engineers tốn nhiều tuần chỉ để "tune learning rate" (điều chỉnh η) sao cho mô hình hội tụ nhanh nhất mà không bị văng khỏi quỹ đạo.

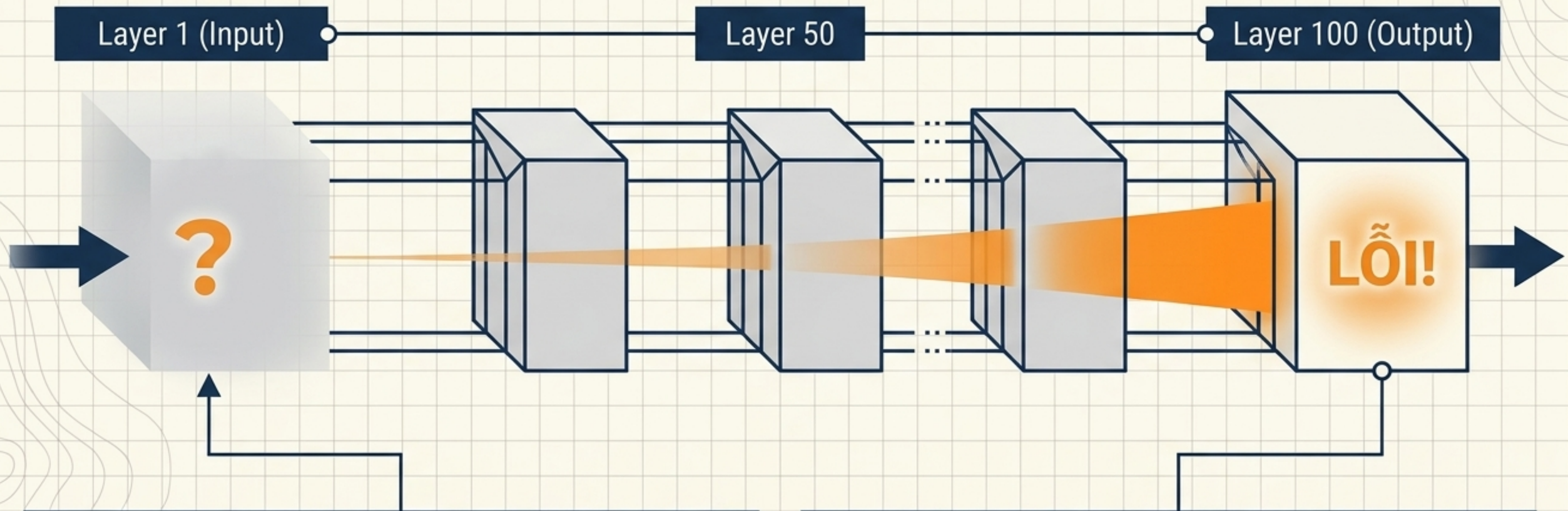
TRADE-OFF 2: CẠM BẦY CỦA ĐỊA HÌNH PHỨC TẠP



GIẢI THÍCH (EXPLANATION)

- **Cực tiểu cục bộ (Local Minima):** Đạo hàm = 0 nhưng chưa phải đáy thấp nhất. Mô hình tưởng đã tối ưu nhưng thực ra vẫn rất tệ.
- **Điểm yên ngựa (Saddle Points):** Bằng phẳng ở một chiều, nhưng dốc ở chiều khác. Gradient descent thường bị chậm lại ở đây.
- **Giải pháp hiện đại:** Sử dụng thuật toán **Adam** hoặc **SGD + Momentum** (tạo đà/quán tính) để “lăn” vọt qua các hố cạn.

THE SCALING PROBLEM: ÁCH TẮC TRUYỀN TIN TRONG MẠNG NƠ-RON SÂU



- **Vấn đề:** Lỗi được đo ở lớp đầu ra (Output). Làm sao lớp đầu tiên (Input) cách đó 100 tràm biết nó cần tự điều chỉnh thế nào?

- **Hạn chế lịch sử:** Nếu tính toán đạo hàm theo kiểu "thử lại từ đầu" cho từng biến một, thời gian chạy sẽ là $O(N^2)$ — bất khả thi về mặt tính toán đối với mạng sâu.

MECHANISM: QUY TẮC CHUỖI (CHAIN RULE) & DÒNG CHẢY GRADIENT

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial Y} \cdot \frac{\partial Y}{\partial X}$$



- Bí ẩn Toán học

Sự thay đổi của X lên Loss bằng TÍCH các thay đổi cục bộ dọc theo đường ống.

- Tính hiệu quả

Mỗi lớp chỉ cần tự đo 'đạo hàm cục bộ' của riêng nó, sau đó nhân với tín hiệu truyền về từ lớp sau.

- Đột phá

Đưa độ phức tạp tính toán từ $O(N^2)$ xuống $O(N)$. Chìa khóa để đào tạo mạng tỉ tham số.

AI APPLICATION: LAN TRUYỀN NGƯỢC (BACKPROPAGATION) TRONG THỰC TẾ

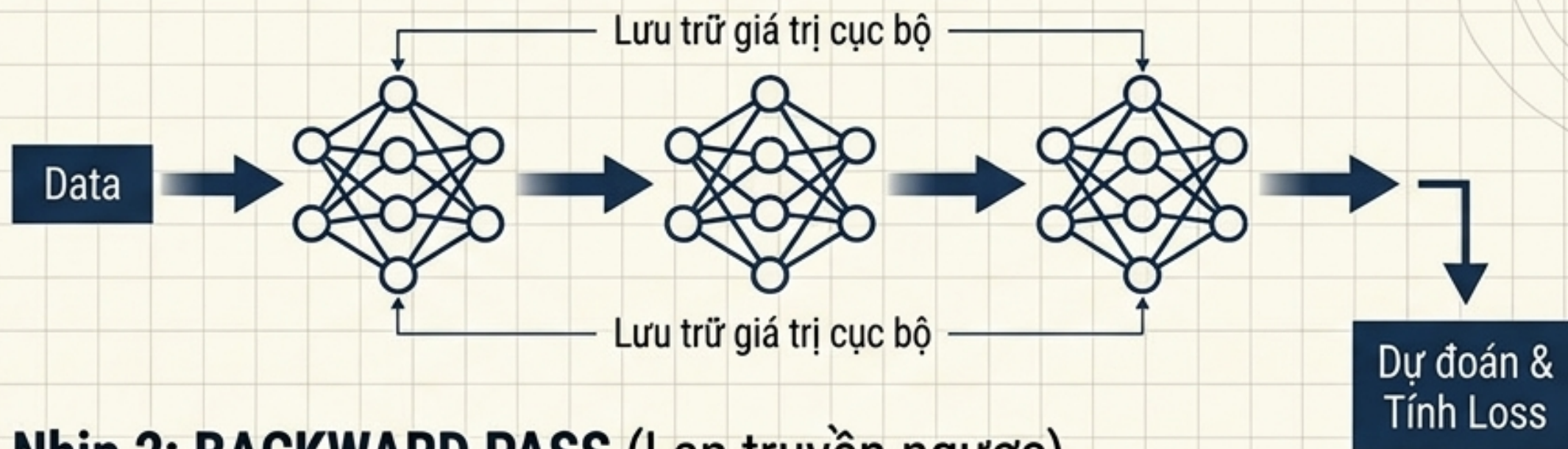
- Ví dụ Google Translate:

Dịch một từ sai (Forward) ->
Đo lỗi -> Lỗi chảy ngược về
từng lớp từ vựng (Backward)
để tự động sửa hàng triệu
trọng số cùng lúc.

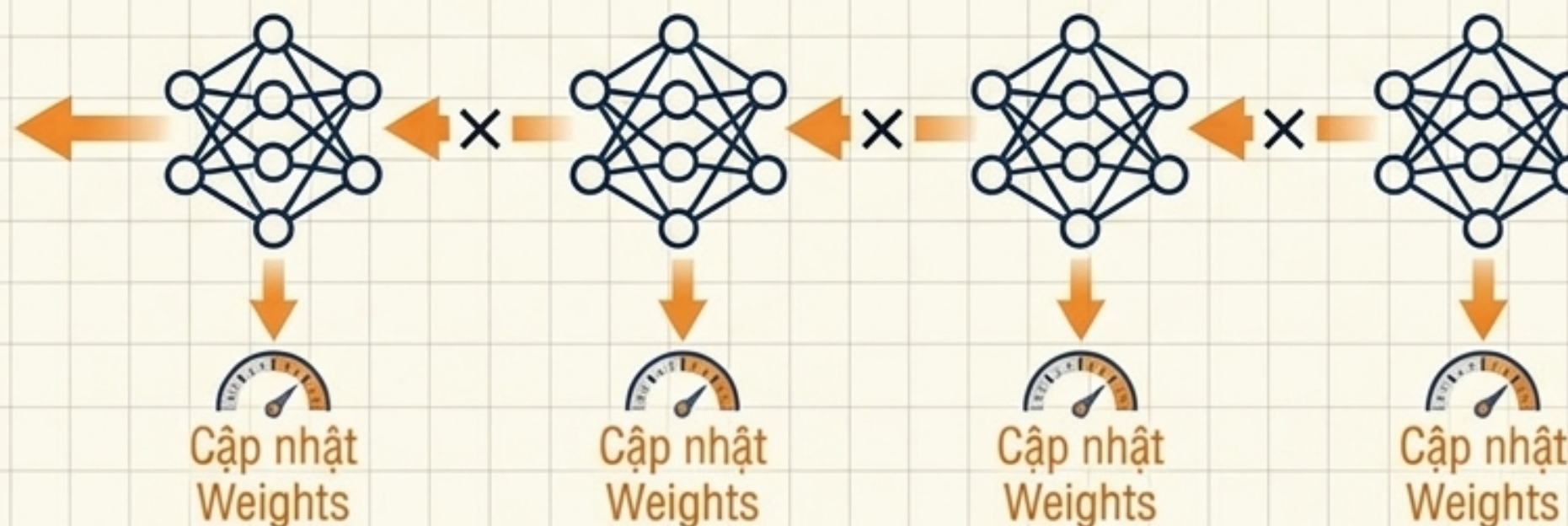
- Thuật toán vĩ đại:

Backpropagation không phải
phép thuật, nó chỉ là Quy tắc
Chuỗi (Chain Rule) được
máy tính thực thi một cách
có hệ thống.

Nhịp 1: FORWARD PASS (Lan truyền thuận)

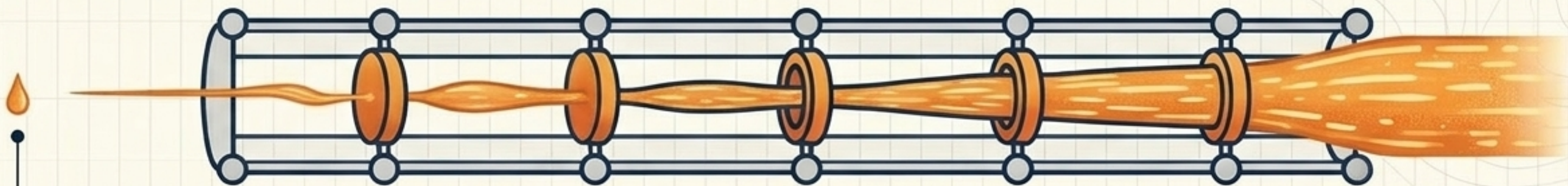


Nhịp 2: BACKWARD PASS (Lan truyền ngược)



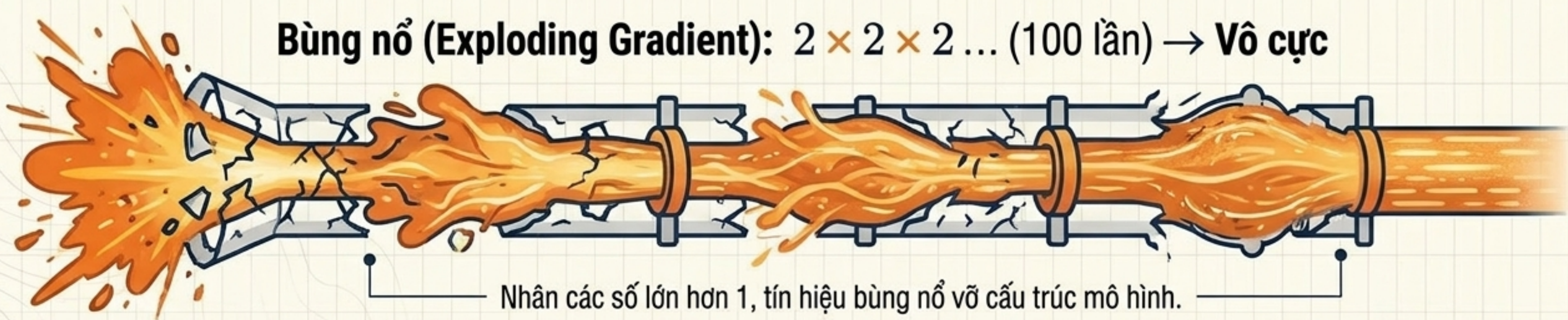
TRADE-OFF 3: BỆNH LÝ “TRIỆT TIÊU” VÀ “BÙNG NỔ” GRADIENT

Triệt tiêu (Vanishing Gradient): $0.5 \times 0.5 \times 0.5 \dots (100 \text{ lần}) \rightarrow 0$



Hệ quả của việc nhân liên tiếp các đạo hàm nhỏ hơn 1. Tín hiệu teo biến về 0, các lớp đầu tiên “chết” (không học được).

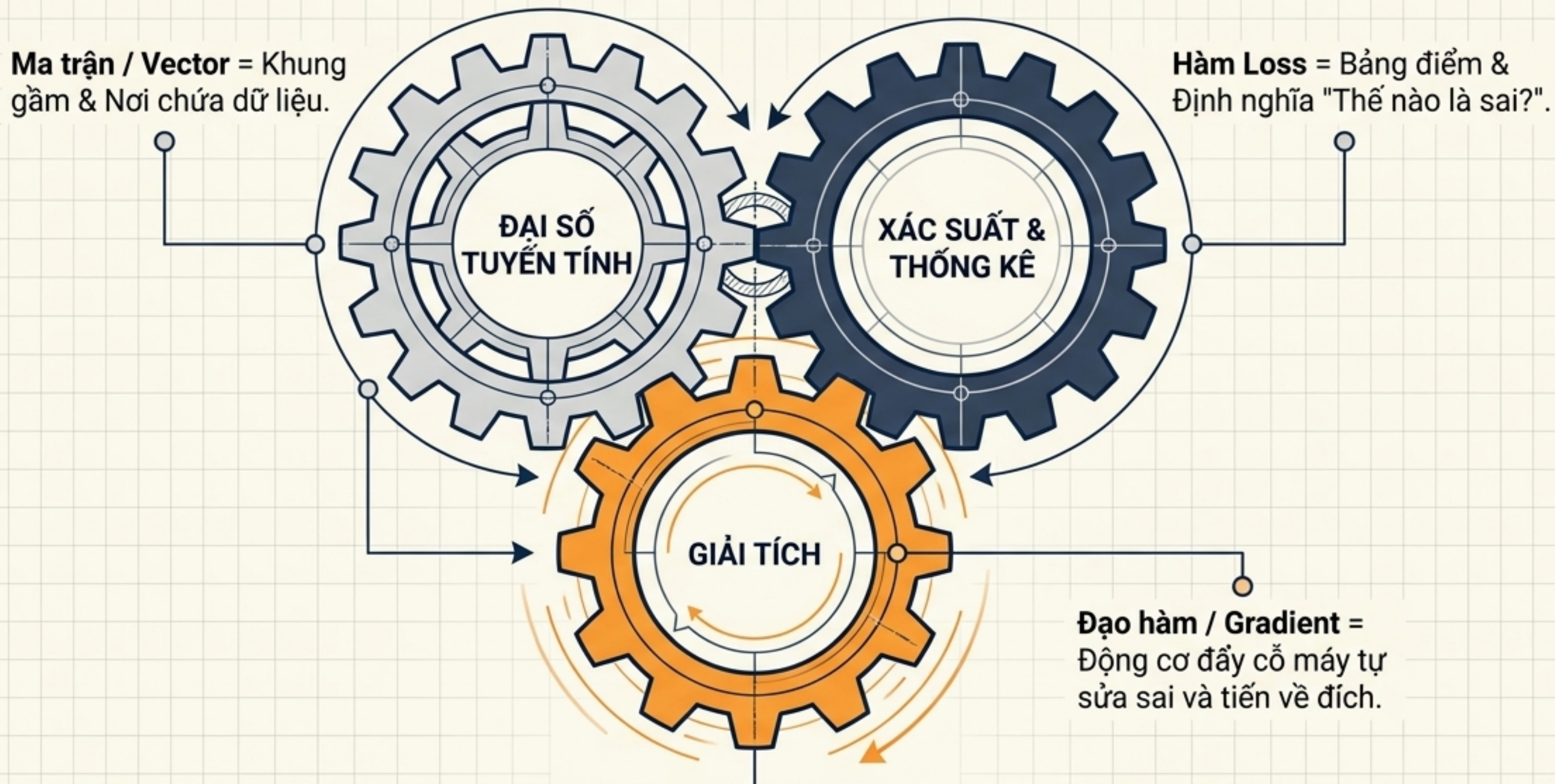
Bùng nổ (Exploding Gradient): $2 \times 2 \times 2 \dots (100 \text{ lần}) \rightarrow \text{Vô cực}$



Nhân các số lớn hơn 1, tín hiệu bùng nổ vỡ cấu trúc mô hình.

- **Kiến trúc định hình bởi Toán học:** Đây là lý do hàn Hàm ReLU thay thế Sigmoid, và kiến trúc ResNet ra đời tạo từ “đường tắt” cho Gradient chảy an toàn.

MENTAL MODEL: CỖ MÁY TOÁN HỌC AI HOÀN CHỈNH



Nhìn lại bức tranh toàn cảnh: Không nhánh toán nào đứng độc lập. Giải tích chính là lực đẩy duy nhất giúp hệ thống AI biết xoay chuyển để tiến về sự hoàn hảo.

THE NEXT STEP: TỪ NỀN TẢNG TOÁN HỌC ĐẾN MACHINE LEARNING

GIAI ĐOẠN 1

Nền tảng Toán học
(Đã hoàn thành)

Động cơ Gradient

GIAI ĐOẠN 2

ML Cổ điển
(Sắp tới)

- Hồi quy tuyến tính
- Logistic
- Trees...

- **Động cơ đã sẵn sàng:** Chúng ta đã có cơ chế tìm đường tối ưu.
- **Chặng tiếp theo:** Lắp động cơ Gradient Descent vào các "khung xe" thuật toán thực tế để giải quyết bài toán: Dự báo giá nhà, Phân loại email, Chấm điểm tín dụng.